Assignment: Database Creation & Loading

CSE 511: Data Processing at Scale

Introduction & Background

In this assignment, you will create a PostgreSQL database from four Reddit CSV files. You will define the tables, load the data, add relationships, and run five SQL queries.

This assignment focuses on the core database workflow: inspect the environment, create tables, load a large dataset, add constraints, and verify the results. We strongly recommend the course Docker image because it provides the tested tools and data and avoids platform-specific setup problems. You may instead deploy an equivalent PostgreSQL environment yourself, but support for self-managed installations is limited.

Dataset Description

The full Pushshift archive is very large. This assignment uses a smaller, curated subset that keeps only the columns needed for the course.

The course dataset contains four CSV files:

1. **submissions:**
    a. The sample below shows the columns and example values.
    b. File: data/submissions.csv. Total number of entries: 1,263,937


```
"downs": 1,
"url": http://thinkprogress.org/economy/2011/06/29/some_url",
"id": "iemqy",
"edited": "False",
"num_reports": "",
"created_utc": 1309564792,
"name": "t3_iemqy",
"title": "A somewhat long title with some political discussion",
"author": "[deleted]",
"permalink": "/r/politics/comments/iemqy/another_string/",
"num_comments": 1,
"likes": "",
"subreddit_id": "t5_2cneq",
"ups": 3
```


2. **comments:**
    a. The sample below shows the columns and example values.
    b. File: data/comments.csv. Total number of entries: 10,557,466

Copyright © Zhichao Cao, Ph.D.

```
"distinguished": "",
"downs": 0,
"created_utc": 1309478400,
"controversiality": 0,
"edited": "False",
"gilded": 0,
"author_flair_css_class": "mordekaiser",
"id": "c22x4aq",
"author": "username",
"retrieved_on": 1427302516,
"score_hidden": "False",
"subreddit_id": "t5_2rfxx",
"score": 1,
"name": "t1_c22x4aq",
"author_flair_text": "[username] (NA)",
"link_id": "t3_id1nc",
"archived": "True",
"ups": 1,
"parent_id": "t3_id1nc",
"subreddit": "leagueoflegends",
"body": "Good lord.  Yes."
```


3. **authors:**
    a. The sample below shows the columns and example values.
    b. File: data/authors.csv. Total number of entries: 6,158,212


```
"id": "t2_1rr1",
"retrieved_on": 1532086586,
"name": "duncan",
"created_utc": 1298437200,
"link_karma": 1029,
"comment_karma": 9943,
"profile_img": "https://www.redditstatic.com/avatars/avatar_default_02_25B79F.png",
"profile_color": "",
"profile_over_18": "False"
```


4. **subreddits**:
    a. The sample below shows the columns and example values.
    b. File: data/subreddits.csv. Total number of entries: 914,067


```
"banner_background_image": "",
"created_utc": 1137700161,
"description": "A very very long description of what the subreddit is about. Shown maybe
only to the members?",
"display_name": "John Doe",
"header_img": "https://b.thumbs.redditmedia.com/h5RmvyztneDL1.png",
```


Copyright © Zhichao Cao, Ph.D.

```
"hide_ads": "False",
"id": "vf2",
"over_18": "True",
"public_description": "Still a description but this one Shown to people not part of the
subreddit",
"retrieved_utc": 1591839904,
"name": "t5_vf2",
"subreddit_type": "public",
"subscribers": 1880887,
"title": "Another Random Subreddit",
"whitelist_status": "all"
```


The relationship diagram for this curated dataset is included in the course materials.

Problem Statement

- Create the four tables authors, subreddits, submissions, and comments, then load each CSV file into its matching table. Use lowercase table and column names that match the CSV headers exactly. Work in the existing database and keep its encoding unchanged.
- Choose appropriate data types, primary keys, unique constraints, and foreign keys.
- Complete Query 1-5 in queries/queries.sql.

Table Creation

Create four empty tables: authors, subreddits, submissions, and comments. Match every table and column name to the CSV headers exactly.

1. **authors table.** Define every column listed in data/authors.csv. Choose suitable PostgreSQL data types and an appropriate primary key.
2. **subreddits table.** Define every column listed in data/subreddits.csv. Choose suitable PostgreSQL data types and an appropriate primary key.
3. **submissions table.** Define every column listed in data/submissions.csv. Choose suitable PostgreSQL data types and an appropriate primary key.
4. **comments table.** Define every column listed in data/comments.csv. Choose suitable PostgreSQL data types and an appropriate primary key.

Data Loading

Load each CSV file into its matching table. You may use PostgreSQL COPY or pg_bulkload. Use one loading method on empty tables and skip the CSV header row.

1. **Load authors.csv.** Load the file into `authors`. The completed table must contain exactly 6,158,212 rows.

Copyright © Zhichao Cao, Ph.D.

2. **Load subreddits.csv.** Load the file into `subreddits`. The completed table must contain exactly 914,067 rows.
3. **Load submissions.csv.** Load the file into `submissions`. The completed table must contain exactly 1,263,937 rows.
4. **Load comments.csv.** Load the file into `comments`. The completed table must contain exactly 10,557,466 rows.

Relationships

Add primary keys, unique constraints, and foreign keys after loading the data. The reference schema includes these relationships:

1. submissions.author → authors(name)
2. submissions.subreddit_id → subreddits(name)
3. comments.author → authors(name)
4. comments.subreddit_id → subreddits(name)
5. comments.subreddit → subreddits(display_name)

Required Queries

Complete Query 1-5 in queries/queries.sql.

1. **Query 1 — Comment count for xymemez.** Count the comments written by xymemez. Return one value named comments count.
2. **Query 2 — Subreddits by type.** Group the subreddits by subreddit_type and count the subreddits in each group. Return the type and its count.
3. **Query 3 — Top subreddits by comment activity.** Group comments by subreddit. Return the subreddit name, its comment count, and its average comment score rounded to two decimal places. Sort by comment count in descending order and return the top 10 rows.
4. **Query 4 — High-karma authors.** Return name, link_karma, comment_karma, and a label for authors whose average karma is greater than 1,000,000. Use label 0 when link karma is greater than or equal to comment karma, and label 1 otherwise. Sort by average karma in descending order.
5. **Query 5 — Comments by subreddit type.** For author [deleted_user], join comments to subreddits and count comments for each subreddit type. Return the subreddit type and comment count in type order.

Important Note:

1. Use the exact four course CSV files. They are included in the Docker image. If you use your own environment, use these same data files; do not substitute another dataset.

Copyright © Zhichao Cao, Ph.D.

2. The sample rows below show the available columns. Some text values are shortened for readability.

Grading

- This is an ungraded practice assignment. The included autograder reports an illustrative score so you can check your work.
- The illustrative 100-point self-check is divided into four parts:
    - **40 points for creating the four required tables.**
    - **20 points for loading the exact number of rows.**
    - 20 points for the required primary keys and 20 points for the results of Query 1-5.
- The self-check scores primary keys but does not score foreign-key choices. It uses the complete course dataset. Course staff evaluate the required files and database/query results, not whether you used Docker or a self-managed environment.

Optional bulk-loading references:

1. dbi Blog (dbi-services.com)
2. http://ossc-db.github.io/pg_bulkload/pg_bulkload.html
3. https://www.postgresql.org/docs/current/populate.html
4. sql - How to speed up insertion performance in PostgreSQL - Stack Overflow
5. 4.pg_bulkload Data Loading Use and Example - www.cqdba.cn - Blog Park (cnblogs.com)
   Note: COPY and pg_bulkload are both valid. There is no execution-time limit and no optimization score.

Copyright © Zhichao Cao, Ph.D.

Appendix: Docker Quick Start

1. How do I start the recommended course container? Docker Hub:
   https://hub.docker.com/r/change1472/cse511-database-creation
   Run: docker pull change1472/cse511-database-creation:latest
   Then run: docker run --name cse511-database-creation-loading -it change1472/cse511-database-creation:latest

2. Do I need to start PostgreSQL manually?
   No. PostgreSQL starts automatically with the container. To return later, run: docker start -ai cse511-database-creation-loading

3. How do I inspect or reset the database?
   The image includes these tools and helper commands:
   (1) ./database.sh info — show PostgreSQL status, tables, and exact row counts
   (2) ./database.sh counts — show exact row counts for all four course tables

   Use ./database.sh shell to open psql. Use ./database.sh reset to remove only the four course tables. To reset and reload, run ./database.sh reload copy or ./database.sh reload pg_bulkload.

4. Where are the CSV files?
   They are in /root/cse511/database-creation-and-loading/data/. Do not download a different copy.

5. What are the details of the database?
   PostgreSQL 14 connection:
   - username: postgres
   - password: postgres
   - Database Name: postgres
   - Host and port: 127.0.0.1:5432

   You do not need to use all the details, only the ones that are necessary

~ The crux of the assignment is to gain the ability to ingest a (somewhat) large amount of data and to understand how that is performed! Make sure that you focus your time on that aspect ~

Copyright © Zhichao Cao, Ph.D.

Sample Answer and Expected Output

The walkthrough below shows one complete Docker-based run. If you use a self-managed PostgreSQL environment, run the same scripts after matching the directory layout and connection settings.

Directory Structure

```
/root/cse511/database-creation-and-loading/
├── README.md                       # Quick reference for the included environment
├── assignment.sh                   # tables, relations, copy, pg_bulkload, all
├── autograde.sh                    # Read-only illustrative 100-point self-check
├── database.sh                     # Status, inspection, reset, reload, and psql shell
├── run_queries.sh                  # Runs Query 1-5
├── data/
│   ├── authors.csv                 # 6,158,212 author records
│   ├── subreddits.csv              # 914,067 subreddit records
│   ├── submissions.csv             # 1,263,937 submission records
│   ├── comments.csv                # 10,557,466 comment records
│   ├── README.md                   # Dataset notes
│   └── SHA256SUMS                  # Checksums for validating the CSV files
├── schema/
│   ├── create_tables.sql           # Creates the four empty tables
│   └── create_relations.sql        # Adds keys and relationships
├── loaders/
│   ├── load_copy.sh                # Loads data with PostgreSQL COPY
│   └── load_pg_bulkload.sh         # Loads data with pg_bulkload
├── queries/
│   └── queries.sql                 # Query 1-5
├── answers/
│   └── expected_results.md         # Reference query results
├── logs/
│   ├── copy/                       # COPY logs
│   └── pg_bulkload/                # pg_bulkload logs
└── output/
    └── query_results.txt           # Created by run_queries.sh
```

The four CSV files, the schema scripts, both loaders, Query 1-5, and the reference results are already included in the image. The logs and query output directories are populated as the scripts run.

Start or Reopen the Container

```
# Download the current course image.
docker pull change1472/cse511-database-creation:latest

# Create the container. PostgreSQL starts automatically, and the shell opens
# in /root/cse511/database-creation-and-loading/.
docker run --name cse511-database-creation-loading -it
change1472/cse511-database-creation:latest
```

Copyright © Zhichao Cao, Ph.D.

```
# Reopen the same container later. Do not run docker run again with the same name.
docker start -ai cse511-database-creation-loading
```

You do not need to start postgresql or use systemctl. This container does not run a full systemd session; its startup script starts PostgreSQL for you.

Inspect and Manage the Database

```
# Confirm that PostgreSQL is ready and show the database, user, and version.
./database.sh status

# List all public tables with estimated row counts and disk sizes.
./database.sh tables

# Show exact row counts for the four course tables.
./database.sh counts

# Run status, tables, and counts together.
./database.sh info

# Open an interactive psql session. Type \dt to list tables and \q to exit.
./database.sh shell

# Drop only the four course tables after confirmation.
# PostgreSQL and the CSV files are kept.
./database.sh reset

# Reset and rebuild with COPY, then add relationships.
./database.sh reload copy

# Reset and rebuild with pg_bulkload, then add relationships.
./database.sh reload pg_bulkload
```

Run the Assignment Step by Step

The main runner accepts exactly five commands:

```
# 1. Recreate the four empty course tables.
./assignment.sh tables

# 2. Load all four CSV files with PostgreSQL COPY.
# Run tables first.
./assignment.sh copy

# 3. Alternative loader: load all four CSV files with pg_bulkload.
# Run tables first. Do not run both loaders on the same nonempty tables.
./assignment.sh pg_bulkload

# 4. Add primary keys, unique constraints, and foreign keys.
# Run this after loading for the fastest workflow.
./assignment.sh relations
```

Copyright © Zhichao Cao, Ph.D.

```
# 5. Fastest complete demonstration: tables -> COPY -> relations.
./assignment.sh all
```

COPY and pg_bulkload are two accepted loading methods. The default all command uses COPY. Loading before adding relationships is faster because PostgreSQL does not need to maintain every constraint while millions of rows are inserted. Students may reverse the order for comparison, but the demonstration workflow uses the faster order.

Run Queries and the Illustrative Self-Check

```
# Run Query 1-5 and save the complete output in output/query_results.txt.
./run_queries.sh

# Run the read-only illustrative 100-point self-check.
# No loader argument is accepted, and there is no execution-time limit.
./autograde.sh
```

Example Complete Run

On a fresh container, the following four commands demonstrate the complete assignment. The reset command is harmless here and makes the starting state explicit. If the tables do not exist yet, you may begin directly with ./assignment.sh all.

```
# Start from empty course tables.
./database.sh reset --yes

# Create tables, load the CSV files with COPY, and add relationships.
./assignment.sh all

# Execute Query 1-5.
./run_queries.sh

# Verify tables, row counts, primary keys, and query results.
./autograde.sh
```

Expected Output

The exact progress messages and execution time may differ. The database readiness, exact row counts, query checks, and final score should match the values below.

Database status

```
PostgreSQL: ready at 127.0.0.1:5432
database=postgres
user=postgres
server_version=14.24 (Ubuntu 14.24-0ubuntu0.22.04.1)
```

Exact row counts

Copyright © Zhichao Cao, Ph.D.

```
table            exact rows
---------------- ----------
authors              6158212
subreddits            914067
submissions          1263937
comments            10557466
```

Query checks

```
Query 1 expected value: 18
Query 2 returned rows:  6
Query 3 returned rows: 10
Query 4 returned rows: 42
Query 5 returned rows:  2
```

Illustrative autograder summary

```
Table creation:       40/40
Data loading:         20/20
Integrity checks:     20/20
Query results:        20/20
------------------------------
Total:               100/100
```

This score is illustrative because the assignment is ungraded. It shows how the former rubric evaluated a complete submission and gives you a quick way

Copyright © Zhichao Cao, Ph.D.